

QA Bug Report — SprintCheck

Comprehensive functional, non-functional, security, accessibility & UX assessment

TARGET	DATE	TESTER
sprint-check-v2-9472.vercel.app	22 Aug 2026	Senior QA (automated + manual, Playwright/Chromium)
BUILD		
Next.js on Vercel		

1. Executive Summary

All 15 public routes were reachable (HTTP 200) with strong performance and clean responsive behaviour. Testing covered happy & negative paths, form validation, CTA/link integrity, page routing, security, accessibility and performance. **12 issues** were found, the most serious being a **non-functional contact form** that silently drops enquiries, and auth-endpoint weaknesses (**account enumeration** and **no rate limiting**) that matter for an identity/KYC product.

3	4	5	11
High	Medium	Low	Checks passed

Top priorities: (1) BUG-01 Contact form loses submissions · (2) BUG-02 Password-reset account enumeration · (3) BUG-03 No auth rate limiting · (4) BUG-05 Sandbox "verifies" empty input.

2. Scope & Method

Routes tested: /, /contact, /docs (+ /bvn-check, /nin-check, /voters-check, /facial-check, /sdk-flow), /privacy, /terms, /sandbox, /sdks, /signin, /signup, /forgot-password, plus 404 and protected-route redirects (/dashboard).

Techniques: real headless-Chromium automation (Playwright) capturing console/page/network errors; form negative-path fuzzing; endpoint probing (curl); responsive testing at 375px & 768px; keyboard/focus & tab-order checks; WCAG contrast sampling; performance timing (TTFB/FCP/LCP/transfer); header & redirect inspection.

3. Findings at a Glance

ID	Severity	Area	Summary
BUG-01	High	Functional / Data loss	Contact form is a non-functional stub — submissions are never sent anywhere
BUG-02	High	Security	Password-reset endpoint enables account enumeration & leaks framework internals
BUG-03	High	Security	No rate limiting / lockout on login or password-reset

ID	Severity	Area	Summary
BUG-04	Medium	Functional / Content	10 dead navigation & footer links (href="#" / no href)
BUG-05	Medium	Functional / Trust	Sandbox "Send request" returns "Verified Successfully" for empty input and never calls the API
BUG-06	Medium	UX / Auth flow	Post-signup redirect to /signin?registered=1 shows no confirmation
BUG-07	Medium	Security / Headers	Missing baseline security response headers
BUG-08	Low	Metadata / SEO	/sandbox uses the generic homepage <title> and meta description
BUG-09	Low	Security / Info leak	/api/dashboard 401 fires on public marketing pages
BUG-10	Low	Validation / Messaging	Signup phone validation shows the wrong error for non-empty invalid input
BUG-11	Low	Accessibility	"Most popular" pricing badge fails WCAG AA contrast
BUG-12	Low	Accessibility	Cookie-consent banner is not dismissible by keyboard (Esc) & icon buttons lack labels

4. Detailed Findings

BUG-01 High

Functional / Data loss

Contact form is a non-functional stub — submissions are never sent anywhere

The "Send message" form on **/contact** shows a "Message sent" success state but makes **no network request** of any kind. Submitted enquiries are silently discarded — no email, no API call, no persistence.

STEPS TO REPRODUCE

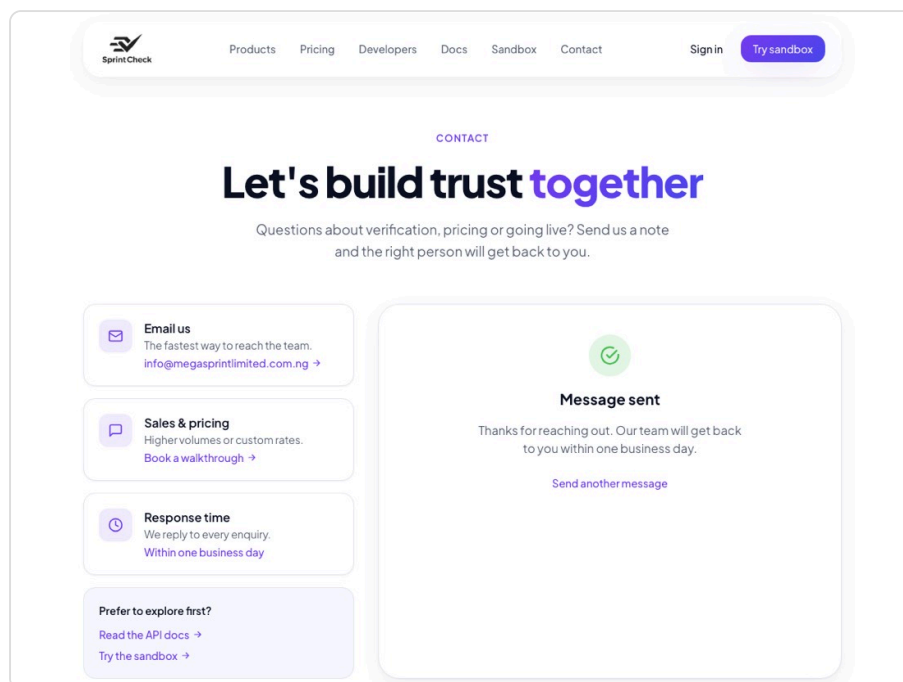
1. Open **/contact**
2. Fill Full name = " " (spaces), Work email = "x@y", Message = "hi"
3. Click "Send message"
4. Observe the DevTools Network tab

EXPECTED

A POST to a backend (or mail service); a validation error for the whitespace name and malformed email.

ACTUAL

UI immediately shows "Message sent ... within one business day". Zero network requests are made. Whitespace name, an email with no TLD ("x@y"), and a 2-char message are all accepted.



Invalid input ("x@y", blank name, "hi") still yields a "Message sent" confirmation.

Password-reset endpoint enables account enumeration & leaks framework internals

POST `/api/auth/forgot-password` returns HTTP **422** with `{"message":"The selected email is invalid."}` for an **unregistered** email. A registered email returns a different (success) response, so an attacker can enumerate which emails have accounts. The wording is also a raw Laravel validation string, disclosing the backend stack.

STEPS TO REPRODUCE

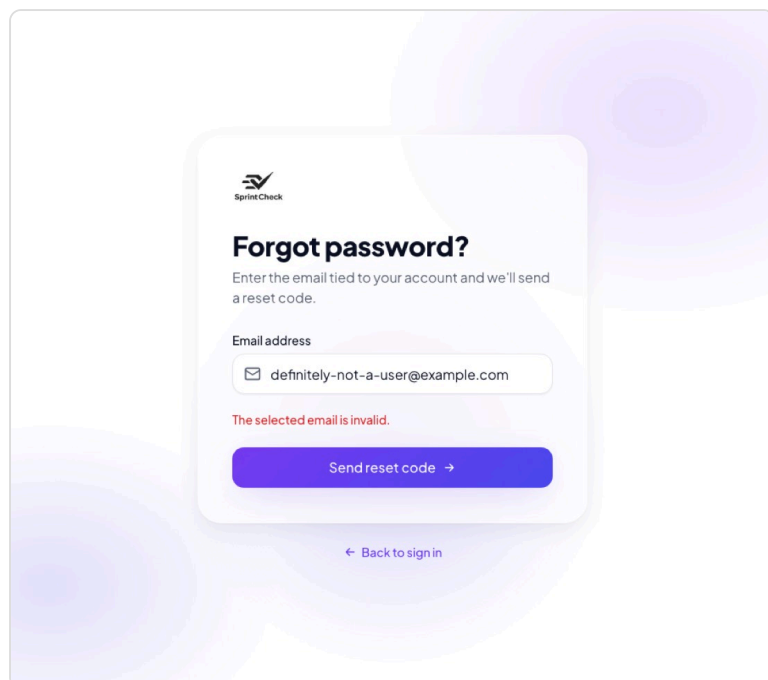
1. Open `/forgot-password`
2. Enter an email you know is NOT registered → note "The selected email is invalid."
3. Enter a registered email → note the different response

EXPECTED

A generic, identical response regardless of whether the email exists (e.g. "If an account exists, we've sent a reset code").

ACTUAL

Distinct responses reveal account existence; raw ORM validation text is exposed to the user.



Unregistered email is explicitly rejected — confirming (by difference) which emails are valid.

No rate limiting / logout on login or password-reset

Six rapid failed logins against `/api/auth/login` all returned 401 with no throttling, CAPTCHA, backoff or logout. Five rapid `/api/auth/forgot-password` calls all processed. This exposes both endpoints to credential brute-forcing and reset-code/email flooding.

STEPS TO REPRODUCE

1. Submit 6 wrong-password logins in quick succession for the same email
2. Submit 5 forgot-password requests in quick succession

EXPECTED

Throttling after a few attempts (429 / temporary logout / CAPTCHA).

ACTUAL

All attempts processed normally (401 / 422). No 429, no delay, no logout.

10 dead navigation & footer links (href="#" / no href)

On every page, these links do nothing: **Banking, Lending, Crypto, Insurance, Gig economy** (Solutions) and **Changelog, About, Careers, Press, Partners** (Company/Developers). They point to "#" or have no href, so clicking scrolls to top / no-ops.

STEPS TO REPRODUCE

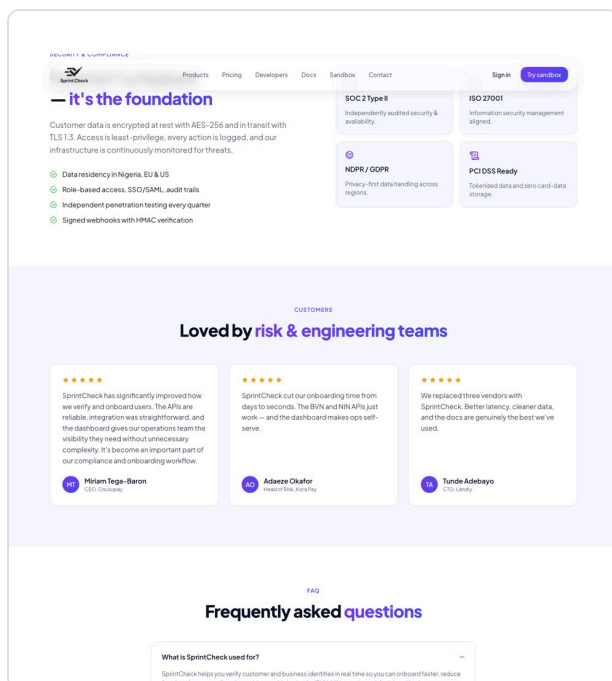
1. Open / (or any page)
2. Scroll to footer
3. Click e.g. "Careers" or "Banking"

EXPECTED

Navigate to the relevant page, or hide the link until the page exists.

ACTUAL

Link is inert (href="#" / missing). 10 such links site-wide.



Dead links highlighted in red across the footer/solutions nav.

Sandbox "Send request" returns "Verified Successfully" for empty input and never calls the API

On `/sandbox`, clicking "Send request" with **all fields blank** (no API key, no BVN, no identifier) returns a full `"success":1, "Verified Successfully"` payload. The response is a hardcoded mock; no network request is made and inputs are ignored entirely.

STEPS TO REPRODUCE

1. Open `/sandbox` (Verify BVN endpoint)
2. Leave API KEY, BVN and IDENTIFIER empty
3. Click "Send request"

EXPECTED

Client-side validation for required fields, or a mocked error path for missing/invalid input, so the sandbox reflects real API behaviour.

ACTUAL

Always returns the same masked "Verified Successfully" record regardless of input; button is never disabled and no request is sent.

The screenshot shows the SprintCheck API Sandbox interface. At the top, there's a navigation bar with links: Products, Pricing, Developers, Docs, Sandbox, Contact, Sign in, and Try sandbox. Below this, a banner says "Try the SprintCheck API live" with a subtext: "Pick an endpoint, send a test request and inspect the JSON response — no signup required. Sandbox returns deterministic, realistic data so you can build with confidence." The main form is titled "Request" and has a "Sandbox" tab. It contains fields for: ENDPOINT (dropdown menu showing "Verify BVN"), METHOD (dropdown menu showing "POST"), API KEY (text input with placeholder "scb_sandbox_demo_XXXXXXXXXXXXXXXXXXXX"), BANK VERIFICATION NUMBER (text input with placeholder "number" and value "22454678613"), and IDENTIFIER (text input with placeholder "identifier" and value "dev@yourapp.com"). There's a "Use sample values" link and a "Send request" button. To the right, a "Response" preview shows a JSON object:

```
{  "success": 1,  "message": "Verified Successfully",  "data": {    "firstName": "SA*****",    "lastName": "OD*****",    "middleName": "AD*****",    "dateOfBirth": "12-Dec-1996",    "phoneNumber1": "0816*****",    "gender": "Male",    "stateOfOrigin": "Osun State",    "bvn": "2245*****",    "base64Image": "/9j/4AAQSkZJRgABAQEAYABgAAD..."  } }
```

Empty request still returns a fully "Verified Successfully" BVN record.

Post-signup redirect to /signin?registered=1 shows no confirmation

A successful signup redirects to `/signin?registered=1`, but the sign-in page renders identically to a normal visit — no "Registration successful, please sign in" banner. Users get no confirmation their account was created.

STEPS TO REPRODUCE

1. Complete /signup with valid details
2. Observe redirect to /signin?registered=1

EXPECTED

A visible success/confirmation message keyed off ?registered=1.

ACTUAL

No message shown; the query param has no visible effect.

Missing baseline security response headers

Responses lack **Content-Security-Policy**, **X-Frame-Options** (clickjacking), **X-Content-Type-Options: nosniff**, **Referrer-Policy** and **Permissions-Policy**. Only HSTS is present. For an identity/KYC product these are expected hardening controls.

STEPS TO REPRODUCE

1. `curl -D - https://sprint-check-v2-9472.vercel.app/ (and /signin)`

EXPECTED

CSP, X-Frame-Options/frame-ancestors, X-Content-Type-Options, Referrer-Policy, Permissions-Policy present.

ACTUAL

Only strict-transport-security is set.

/sandbox uses the generic homepage <title> and meta description

The sandbox page's document title is "SprintCheck — Verify identity & customer data in seconds" (the homepage default) instead of a sandbox-specific title. Its meta description is also the homepage copy.

STEPS TO REPRODUCE

1. Open /sandbox
2. Inspect document.title /

EXPECTED

A page-specific title, e.g. "Interactive API Sandbox — SprintCheck".

ACTUAL

Homepage title/description reused.

BUG-09

Low

Security / Info leak

/api/dashboard 401 fires on public marketing pages

Every public page (/ , /contact, /privacy, /terms, /sdks, /sandbox, 404) issues a client request to `/api/dashboard` that returns 401 ("Your session has expired..."), producing a console error for logged-out visitors and unnecessary requests.

STEPS TO REPRODUCE

1. Open / while logged out
2. Watch console / network → 401 /api/dashboard

EXPECTED

Dashboard data fetched only on authenticated dashboard routes.

ACTUAL

401 request on every public page; console error on load.

BUG-10

Low

Validation / Messaging

Signup phone validation shows the wrong error for non-empty invalid input

On /signup, entering a non-numeric phone like "abcdefg!!!" produces "Enter your phone number." (the empty-field message) rather than the format message "Enter a valid phone number (7–15 digits)." shown for other invalid values.

STEPS TO REPRODUCE

1. Open /signup
2. Phone = "abcdefg!!!", fill other fields, submit

EXPECTED

"Enter a valid phone number (7–15 digits)."

ACTUAL

"Enter your phone number." (implies the field is empty).

BUG-11

Low

Accessibility

"Most popular" pricing badge fails WCAG AA contrast

The "Most popular" badge uses purple text `rgb(110,60,240)` on a ~6% purple tint over white (effectively near-white). Measured contrast $\approx 4.0:1$ at 12px — below the 4.5:1 AA minimum for normal text.

STEPS TO REPRODUCE

1. Open / → Pricing section → "Most popular" badge

EXPECTED

$\geq 4.5:1$ (darken text or the chip background).

ACTUAL

$\approx 4.0:1$.

Cookie-consent banner is not dismissible by keyboard (Esc) & icon buttons lack labels

The cookie banner does not close on Escape. Separately, several icon-only buttons (e.g. header/menu, an unnamed button on /signin & /signup, sandbox controls) expose empty accessible names to screen readers.

STEPS TO REPRODUCE

1. Open / → press Esc on the cookie banner (stays open)
2. Inspect buttons with empty innerText/aria-label

EXPECTED

Esc dismisses the dialog; every interactive control has an accessible name.

ACTUAL

Banner persists; multiple buttons have no accessible name.

5. What Works Well (Passed Checks)

- Responsive layout: no horizontal scroll at 375px (mobile) or 768px (tablet) across all 8 tested pages.
- Performance: TTFB $\approx$ 190 ms, FCP < 450 ms, total transfer 240–354 KB, no single asset > 150 KB.
- Login returns a generic "Invalid credentials" (no enumeration) and password reset does not auto-log-in.
- Signup happy path works: account is created and user is redirected to sign-in.
- Form field labelling: nearly all inputs have associated s and correct autocomplete attributes.
- Signup client validation is thorough (required fields, email format, min-8 password, password match, terms).
- Show/hide password toggle works; focus outlines are visible; tab order on /signin is logical.
- Cookie consent choice persists across reload (localStorage sc-cookie-consent).
- HTTP→HTTPS (308) and trailing-slash (308) redirects behave correctly; 404 page is styled and correct.
- External partner links open in new tabs and all resolve (HTTP 200); most use rel="noopener".
- All images have alt attributes; no broken images detected on any page.

6. Suggestions & Recommendations

1. Wire the contact form to a real backend/email service and add server-side validation; trim & reject whitespace-only names and require a proper email (TLD).
2. Make password-reset responses uniform ("If an account exists...") and replace raw ORM validation strings with product copy.
3. Add rate limiting + exponential backoff (and consider CAPTCHA) on /api/auth/login and /api/auth/forgot-password.
4. Add security headers via next.config / middleware: CSP, X-Frame-Options (or frame-ancestors), X-Content-Type-Options, Referrer-Policy, Permissions-Policy.
5. Either build out or hide the 10 placeholder nav/footer links until their destinations exist.
6. Give the sandbox required-field validation and a mocked error path so it mirrors real API responses; disable "Send request" until inputs are present.
7. Show a success banner on /signin?registered=1 and consider adding server-side password-strength rules to match the client.
8. Publish robots.txt and sitemap.xml (both currently 404) and give /sandbox its own